SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I s.prajith kumar(1912260177)_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

s.prajith

Annexure A

Scenario Based Assignment

1. **Scenario** : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty ii. Explain how A* would handle the same situation, including how it balances cost and heuristic iii. Analyze the impact of heuristic accuracy on both algorithms iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering realtime constraints, optimality, and safety **1. AI-Powered Emergency Medicine Delivery Using Search Algorithms**

i. Greedy Best-First Search

Greedy Best-First Search selects the next location based only on the heuristic value: [f(n)=h(n)]

where (h(n)) is the estimated straight-line distance from the current location to the medicine delivery point.

In the flood-affected region, the drone will prefer the path that appears geographically closest to the destination. It can quickly find a route and is

therefore useful when fast decision-making is more important than finding the minimum-cost route.

Strengths:

- Fast and computationally simple.
- Uses the available distance heuristic.
- Suitable for urgent deliveries.
- Can quickly select a promising direction.

Weaknesses:

- Does not consider the actual travel cost already incurred.
- May select a flooded or risky route because it appears shorter.
- Can produce a non-optimal route.
- Inaccurate heuristic information can lead to poor decisions.

For example, a road that is geographically close may be blocked by floodwater, while a slightly longer road may be completely safe.

ii. A* Search

A* considers both the actual cost and the estimated remaining cost:

[f(n)=g(n)+h(n)] where:

- (g(n)) = actual cost from the starting point to node (n)
- (h(n)) = estimated cost from \(n\) to the destination
- (f(n)) = total estimated cost of the route

For the drone, (g(n)) can include distance, weather conditions, terrain difficulty, fuel/battery consumption and risk. The heuristic (h(n)) can represent the estimated remaining distance.

Therefore, A* does not simply choose the closest location. It chooses a path that provides a good balance between current cost and future estimated cost.

If the heuristic is admissible, A* can find an optimal path in a static environment.

iii. Impact of Heuristic Accuracy

The quality of the heuristic strongly affects both algorithms.

Algorithm	**Effect of Accurate Heuristic**	**Effect of Inaccurate Heuristic**
Greedy Best-First	Quickly selects promising paths	May select unsafe or poor paths
A*	Finds efficient paths with fewer unnecessary searches	May explore inefficient paths and lose optimality if the heuristic is not admissible

Greedy Search depends heavily on \(h(n)\) because it uses only the heuristic.

A* is generally more robust because it also considers \(g(n)\). However, an overestimated heuristic can cause A* to select a non-optimal route.

iv. Effect of Dynamic Changes

Blocked roads create a **dynamic search environment**. A path that was valid when the search started may become unavailable later.

The drone must therefore:

1. Receive real-time information.
2. Mark blocked paths as unavailable.
3. Recalculate affected routes.
4. Update movement costs according to weather and terrain.
5. Re-plan when necessary.

Repeated complete searches can increase computational time. Therefore, real-time or incremental path-planning techniques can further improve performance.

v. Recommended Algorithm

*A Search is the most suitable basic algorithm** because it considers both actual movement cost and estimated remaining cost.

It provides a better balance between:

Speed

Route quality

Safety

Travel cost

Heuristic information

For a practical emergency system, A* should be combined with real-time replanning whenever roads become blocked.

Conclusion: Greedy Best-First Search is faster but may select risky routes. A* provides better cost-aware decision-making and is therefore preferable when medicine delivery must be both fast and safe.

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

2. Smart City Traffic Signal Optimization Optimization Formulation

The problem can be represented as an optimization problem where the AI searches for signal timings that minimize:

[F=w1W+w2C+w3G]

where:

(W) = total vehicle waiting time

(C) = fuel consumption

(G) = traffic congestion

(w1,w2,w3) = importance weights

The variables can include:

Green-light duration

Red-light duration

Signal phase sequence

Offset between neighboring intersections

The objective is:

[min F]

Why Traditional Search Is Inefficient

Hundreds of intersections create an extremely large search space. Each intersection can have several possible signal configurations. Searching every possible combination would require enormous computational resources.

Additionally, traffic conditions change continuously, so a solution that is optimal at one moment may become unsuitable later.

i. Hill Climbing

Hill Climbing starts with an initial signal configuration and continuously moves to a neighboring configuration that improves the objective function. For example:

Current solution → Increase green time → Evaluate traffic → Keep change if improvement occurs

The process continues until no neighboring solution provides an improvement.

Limitations

Can become trapped in a local optimum.

May fail to find the global optimum.

Performance depends on the initial solution.

Dynamic traffic can make the current solution obsolete.

ii. Local Maxima, Plateaus and Ridges Local Maximum

A signal configuration may be better than all its immediate neighbors but not globally optimal. Real-world

example:
Improving traffic at one junction may temporarily reduce congestion there but increase congestion across nearby junctions.

Plateau

Many neighboring solutions may have approximately the same performance.

Example:
Changing a traffic signal by a few seconds may produce almost no measurable improvement.

The algorithm may not know which direction to take.

Ridge

The best solution may require several coordinated changes rather than one individual change.

Example:
Three connected intersections may need their signal timings changed together. Changing only one signal may appear worse even though changing all three produces a significant improvement.

iii. Simulated Annealing and Genetic Algorithms Simulated Annealing

Simulated Annealing sometimes accepts a worse solution to escape local optima.

At high temperature, the algorithm explores more freely. As the temperature decreases, it becomes more selective.

Thus:

Exploration → Escape local optimum → Gradual convergence

Genetic Algorithm

A Genetic Algorithm maintains a population of possible signal configurations.

Main steps:

1. Generate initial population.
2. Calculate fitness.
3. Select better solutions.
4. Perform crossover.
5. Apply mutation.
6. Generate a new population.
7. Repeat until a satisfactory solution is obtained.

It can explore many possible traffic configurations simultaneously.

iv. Recommended Approach

Genetic Algorithm is recommended for this scenario because the problem contains hundreds of intersections and a very large search space.

It provides:

Good scalability.

Population-based exploration.

Ability to escape local optima.

Adaptability to changing traffic patterns.

Suitable optimization of multiple objectives.

A practical system can use Genetic Algorithm for periodic optimization and real-time traffic feedback for continuous adjustment.

Conclusion: Hill Climbing is simple and fast but can become trapped in local solutions. Genetic Algorithms provide better global exploration and are more suitable for large-scale smart-city traffic optimization.

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search ii. Analyze the challenges of partial observability and dynamic environments iii. Describe how the rover builds and updates its internal model iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety **3. Autonomous Mars Rover – Online Search**

i. Need for an Online Search Agent

An offline search agent assumes that the environment and map are already known before planning.

The Mars rover does not have a complete map. It discovers:

Rocks

Craters

Slopes

Safe paths

New terrain while moving.

Therefore, it must follow the cycle:

Sense → Plan → Act → Learn → Update → Re-plan

This makes it an online search problem.

An online search agent makes decisions using the information currently available and improves its knowledge as it explores.

ii. Challenges of Partial Observability and Dynamic Environment
Partial Observability

The rover cannot observe the entire environment.

It may only detect hazards within the range of its sensors.

Therefore:

Unknown obstacles may exist.

The complete map is unavailable.

Sensor measurements may contain uncertainty.

The rover must make decisions with incomplete information.

Dynamic/Changing Conditions

Although Mars is relatively stable compared with Earth, the rover may experience:

Changing terrain conditions.

Dust affecting sensors.

Wheel slippage.

Unexpected obstacles.

Communication delays.

Energy limitations.

These factors can make a previously planned route unsuitable.

iii. Building and Updating the Internal Model

The rover maintains an internal representation of its surroundings.

A simplified process is:

Sensors → Environmental observations → Internal map → Search/planning → Movement

For example:

1. Sensors detect an obstacle.
2. Rover records its approximate location.
3. The internal map is updated.
4. The path through the obstacle area is marked unsafe.

5. A new route is calculated.
6. The rover moves toward the next safe location.

As more information becomes available, the rover continuously improves its internal map. **iv. Comparison of Search Strategies**

Feature	**Uninformed Search**	**Informed Search**	**Online Search**
Environment knowledge	Usually known	Usually known	Initially incomplete
Heuristic	Not required	Required	Can use available information
Complete map	Usually assumed	Usually assumed	Not required
Adaptation	Low	Moderate	High
Suitable for Mars rover	Limited	Useful	Highly suitable

Uninformed algorithms such as BFS and DFS do not effectively exploit distance or safety information.

Informed algorithms such as A* use heuristic knowledge and can produce efficient paths when a map is available.

Online search is better when the rover must discover the environment while moving.

v. Recommended Approach

An online informed search approach is most suitable.

The rover should:

Use sensor information.

Maintain an internal map.

Use heuristic information such as distance and terrain risk.

Re-plan whenever new hazards are discovered.

Prefer safe routes rather than simply shortest routes.

Conclusion: Online search is essential because the rover cannot depend on a complete map. Combining online exploration with heuristic path planning provides better adaptability, safety and efficiency.

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation ii. Explain how backtracking search works in this scenario iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

4. University Exam Timetabling as a CSP

A **Constraint Satisfaction Problem (CSP)** consists of:

[CSP=(X,D,C)]

where:

(X) = variables

(D) = domains

(C) = constraints

i. Variables, Domains and Constraints Variables

Each exam can be represented as a variable.

For example:

[X={E_1,E_2,E_3,ldots,E_n}]

where each (E_i) represents an examination.

Domains

Each exam can have possible combinations of:

Date

Time slot

Examination hall

For example:

[D(E_1)={(Mon,9AM,H1),(Mon,2PM,H2),(Tue,9AM,H3)}]

Constraints

1. Student Conflict Constraint

Two exams taken by the same student cannot occur at the same time.

[E_i.time neq E_j.time]

2. Hall Capacity

The number of exams scheduled simultaneously cannot exceed available halls.

3. Faculty Availability

An exam must be scheduled only when the required faculty members are available.

4. Precedence Constraint

If Subject A must occur before Subject B:

[Time(A)<Time(B)]

5. Special Accommodation

Students requiring special facilities must be assigned to suitable halls and time slots.

ii. Backtracking Search

Backtracking assigns values to variables one at a time.

Example:

Exam A → Monday 9 AM → Check constraints

If valid:

Exam B → Monday 2 PM → Check constraints

If an assignment violates a constraint, the algorithm goes back and changes the previous assignment.

The process is:

Assign → Check → Continue

or, if invalid:

Backtrack → Change assignment → Continue

This continues until a valid timetable is obtained.

iii. Constraint Propagation and Forward Checking

Forward checking improves backtracking by removing invalid values from the domains of unassigned variables.

For example, if Exam A is assigned: **Monday 9 AM, Hall 1**

then another exam involving the same students cannot use

Monday 9 AM.

That value is removed from its domain.

Thus:

[D(E_B)={Mon,2PM,Tue,9AM,ldots}]

instead of repeatedly trying known-invalid assignments.

Benefits

Reduces search space.

Detects conflicts earlier.

Reduces unnecessary backtracking. • Improves computational efficiency. **iv. Real-World Challenges and Best Strategy**

Real-world challenges include:

Large number of students.

Large number of courses.

Limited examination halls.

Faculty availability.

Student conflicts.

Special accommodations.

Last-minute timetable changes.

Multiple simultaneous constraints.

For scalability, the best strategy is a **CSP solver using backtracking with constraint propagation**, enhanced by heuristics such as:

Minimum Remaining Values (MRV)

Degree heuristic

Least Constraining Value (LCV)

Forward checking

These techniques reduce unnecessary exploration.

Conclusion: CSP is highly suitable because exam timetabling naturally consists of variables, domains and constraints. Constraint propagation combined with heuristic backtracking provides an efficient and scalable solution.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta
Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem ii.

Analyze the computational challenges of large game trees iii.

Explain how Alpha-Beta pruning improves efficiency with

detailed reasoning iv. Design an evaluation function and justify

the factors considered

v. Recommend a strategy for balancing optimality and real-time performance **5. Strategic Game AI – Minimax and Alpha-Beta Pruning**

i. Minimax Model

Minimax models a competitive game involving two players:

MAX = AI player

MIN = human opponent

The AI assumes that the opponent will make the best possible move against it.

The game tree can be represented as:

Current State → Possible Moves → Opponent Moves →

Future States MAX tries to maximize its evaluation score,

while MIN tries to minimize it.

For example:

[Utility(S)= begin{cases} +100 & text{AI wins}\ 0 & text{Draw} -100 & text{AI loses} end{cases}]

The AI selects the move with the highest guaranteed outcome.

ii. Computational Challenges Real-time

strategy games can

have:

Thousands of possible actions.

Very large branching factors.

Long sequences of moves.

Continuously changing game states.

If the branching factor is \(b\) and search depth is \(d\), the approximate number of nodes is: [O(b^d)]

For example, if: [b=10,quad d=6

] then approximately: [

10^6=1,000,000] game states

may need to be examined.

This can exceed the available time for real-time decision-making.

iii. Alpha-Beta Pruning

Alpha-Beta pruning improves Minimax by eliminating branches that cannot influence the final decision.

It maintains two values:

Alpha ((alpha)) = best value MAX can guarantee.

Beta ((beta)) = best value MIN can guarantee.

Pruning occurs when:

[alpha geq beta]

Example

Suppose MAX has already found a move with value:

[alpha=8]

While examining another branch, MIN finds an option that produces:

[beta=6]

Since MIN can force the result to be 6 or lower, MAX would never choose this branch over the already available value of 8.

Therefore, the remaining unexplored nodes of that branch can be **pruned**.

Benefits

Reduces the number of nodes evaluated.

Allows deeper search within the same time.

Produces the same decision as Minimax when pruning is applied correctly.

Improves real-time performance.

With good move ordering, Alpha-Beta can reduce the effective search complexity significantly.

iv. Evaluation Function

Because the complete game tree cannot be searched in real time, the AI uses an evaluation function at a selected depth.

A possible evaluation function is:

[E(s)= w_1M+w_2R+w_3U+w_4P+w_5T+w_6D]

where:

(M) = military/unit strength

(R) = available resources

(U) = territorial/control advantage

(P) = production capability

(T) = technological advantage

(D) = damage/losses

(w_1,ldots,w_6) = weights

A higher evaluation value represents a better position for the AI.

Factors Considered

1. **Unit strength:** More powerful units increase the score.
2. **Resources:** More resources provide greater future possibilities.
3. **Territory:** Controlling important areas provides strategic advantage.
4. **Technology:** Advanced technology can improve future actions.
5. **Production:** Greater production capacity helps generate additional units.
6. **Damage:** Loss of important units reduces the score.

The weights should be tuned according to the game strategy.

v. Recommended Real-Time Strategy The

best approach is:

Depth-Limited Minimax + Alpha-Beta Pruning + Heuristic Evaluation + Good Move Ordering

The AI can operate as follows:

1. Generate possible moves.
2. Order the most promising moves first.
3. Apply Alpha-Beta pruning.
4. Search to a fixed depth.
5. Evaluate non-terminal states using the heuristic function.
6. Select the best move.
7. Repeat when the game state changes.

For strict time limits, iterative deepening can be used:

[Depth=1\rightarrow2\rightarrow3\rightarrow\cdots]

The AI always has a valid move from the deepest completed search when the time limit expires.

Conclusion

Minimax provides rational adversarial decision-making, while Alpha-Beta pruning reduces unnecessary computation. Combining both with heuristic evaluation, move ordering and iterative deepening gives a strong balance between optimality, computational efficiency and real-time performance.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues

Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes wellreasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand